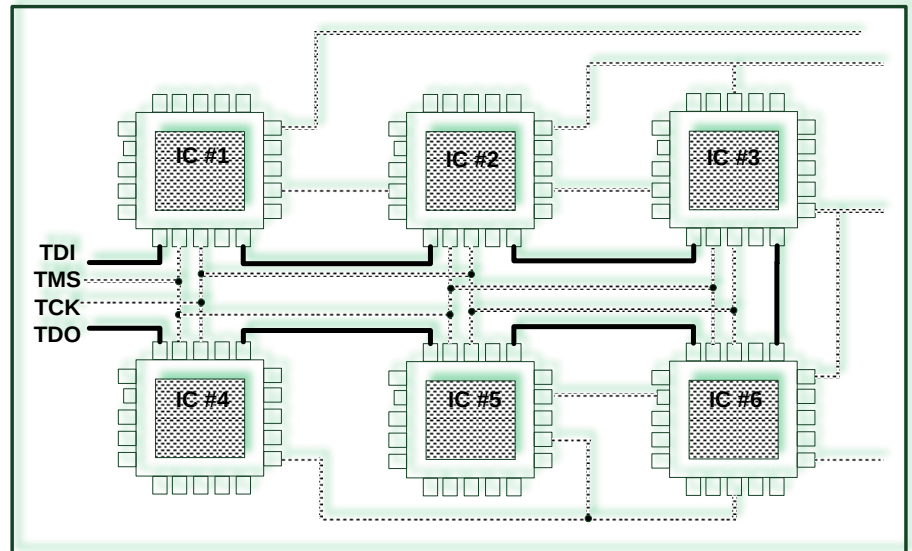


DFT – Part 2

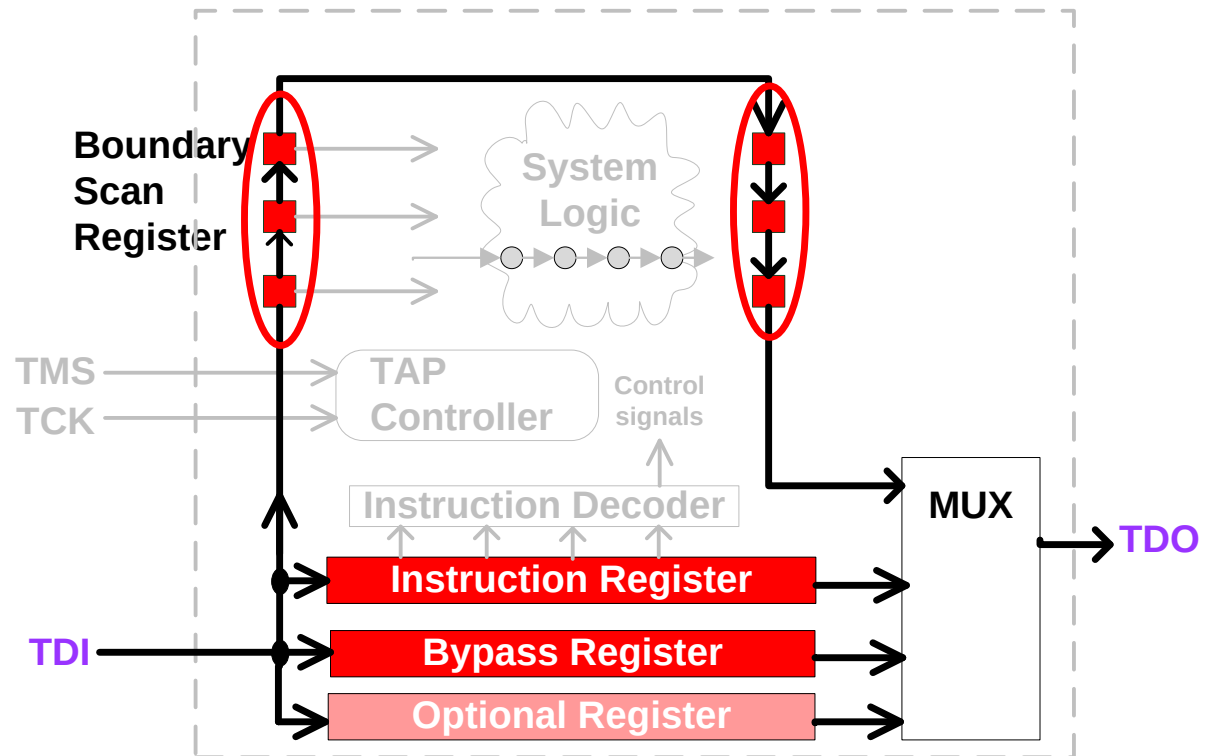
- Introduction
- JTAG Architecture and Components
 - ◆ TAP
 - ◆ TAP controller
 - ◆ Registers
 - * Bypass Register (BR)
 - * Boundary Scan Register (BSR)
 - * Instruction Register (IR)
 - ◆ Instruction Decoder
- JTAG Instructions
- Conclusion



JTAG Registers

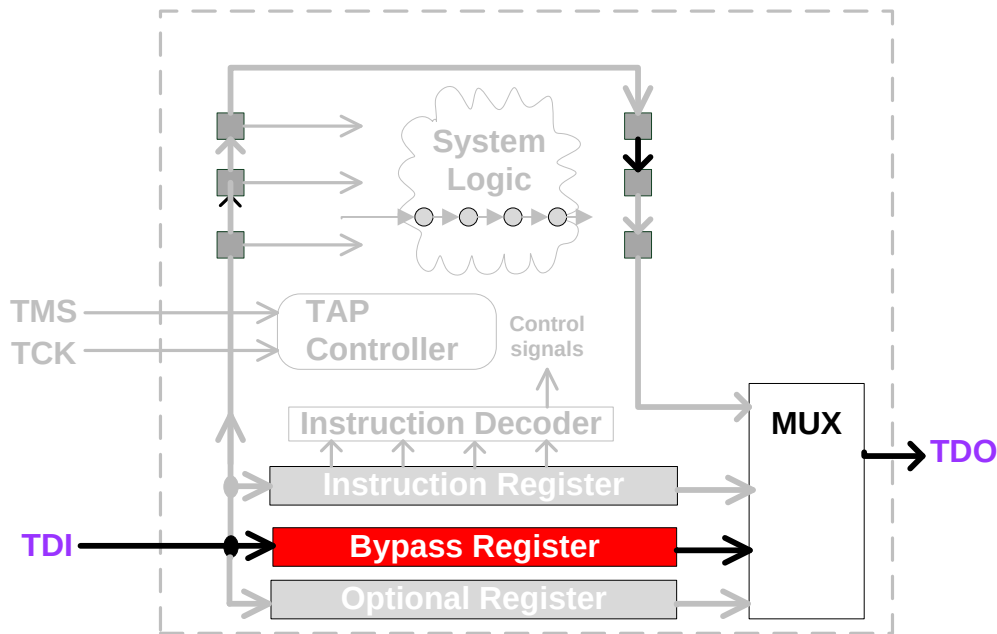
- **Data Register, DR**
 - ♦ **Bypass Register, BR**
 - ♦ **Boundary Scan Register, BSR**
 - **Instruction Register, IR**
- } **mandatory registers**

- **Optional Data Registers**
 - ♦ **Device ID register**
 - ♦ **Device specific register**
 - ♦ (not in lecture)
- Registers share same TDI/TDO

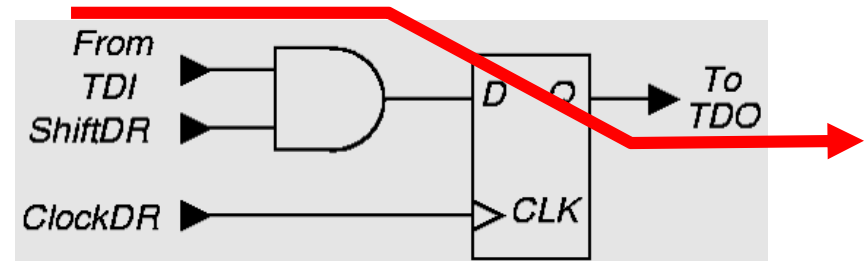


Bypass Register

- **Purpose:** provide short cut from TDI to TDO Ensure consistency when transferring data between different chips.
- **One-bit FF:** shift data from TDI to TDO when **ShiftDR=1**

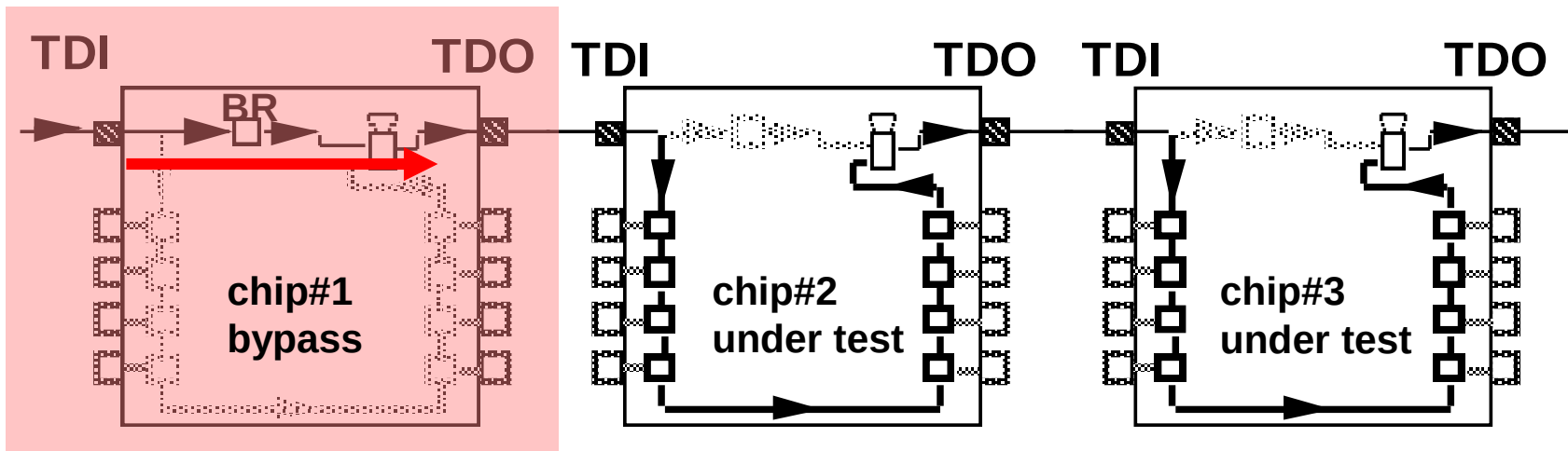


*DR=data register



Bypass Register

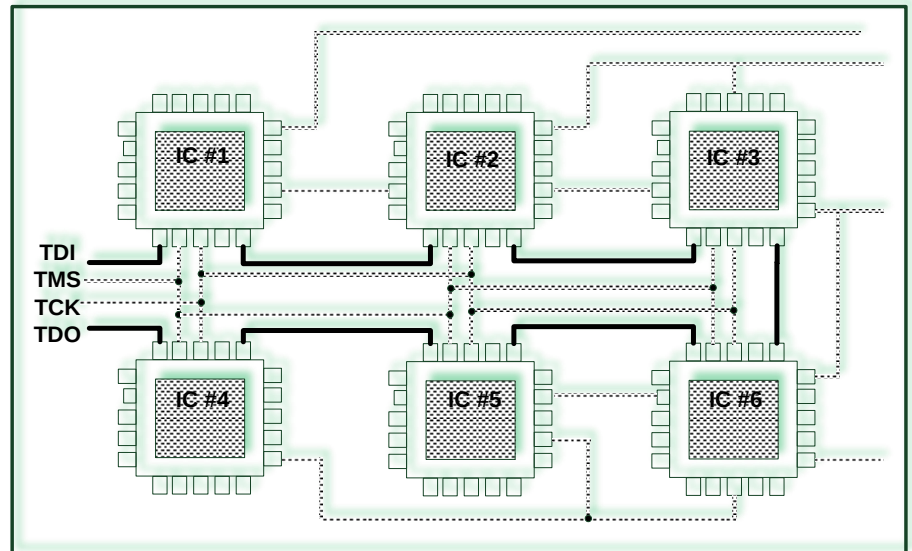
- BR provides a single-bit shortcut through the chip
 - ♦ Shorten boundary scan chain to **chip under test**
- Example: three chips
 - ♦ Go through three chips: 24 clocks
 - ♦ Bypass chip#1: $1+8+8=17$ clocks
 - ♦ Reduce 29% test time



BR Saves Test Time

DFT – Part 2

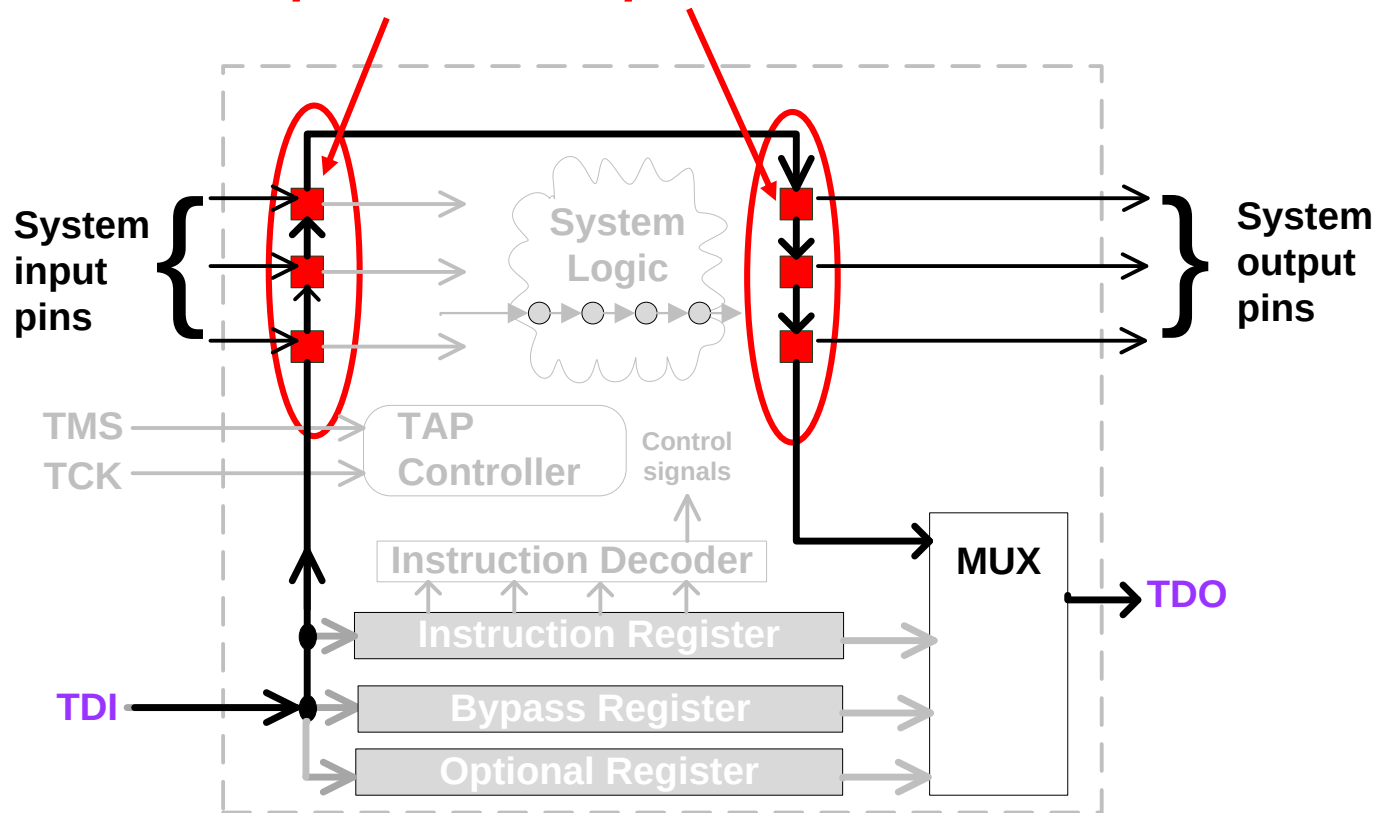
- Introduction
- JTAG Architecture and Components
 - ◆ TAP
 - ◆ TAP controller
 - ◆ Registers
 - * Bypass Register (BR)
 - * Boundary Scan Register (BSR)
 - * Instruction Register (IR)
 - ◆ Instruction Decoder
- JTAG Instructions
- Conclusion



Boundary Scan Registers, BSR

- Purpose : Control system I/O pins; Observe system I/O pins
- BSR consists of **Boundary Scan Cells (BSC)**

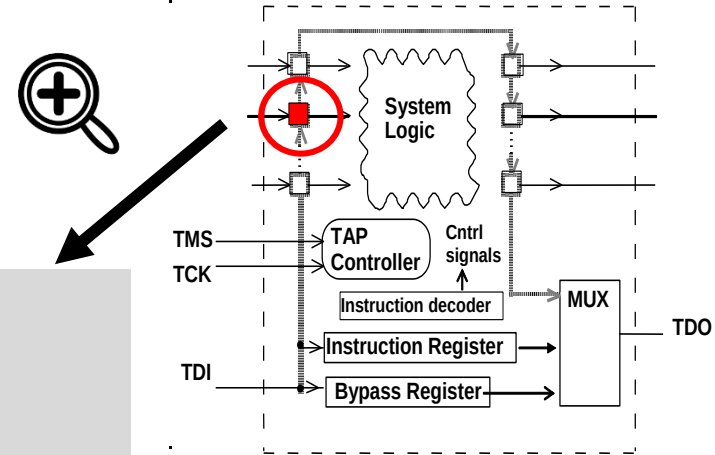
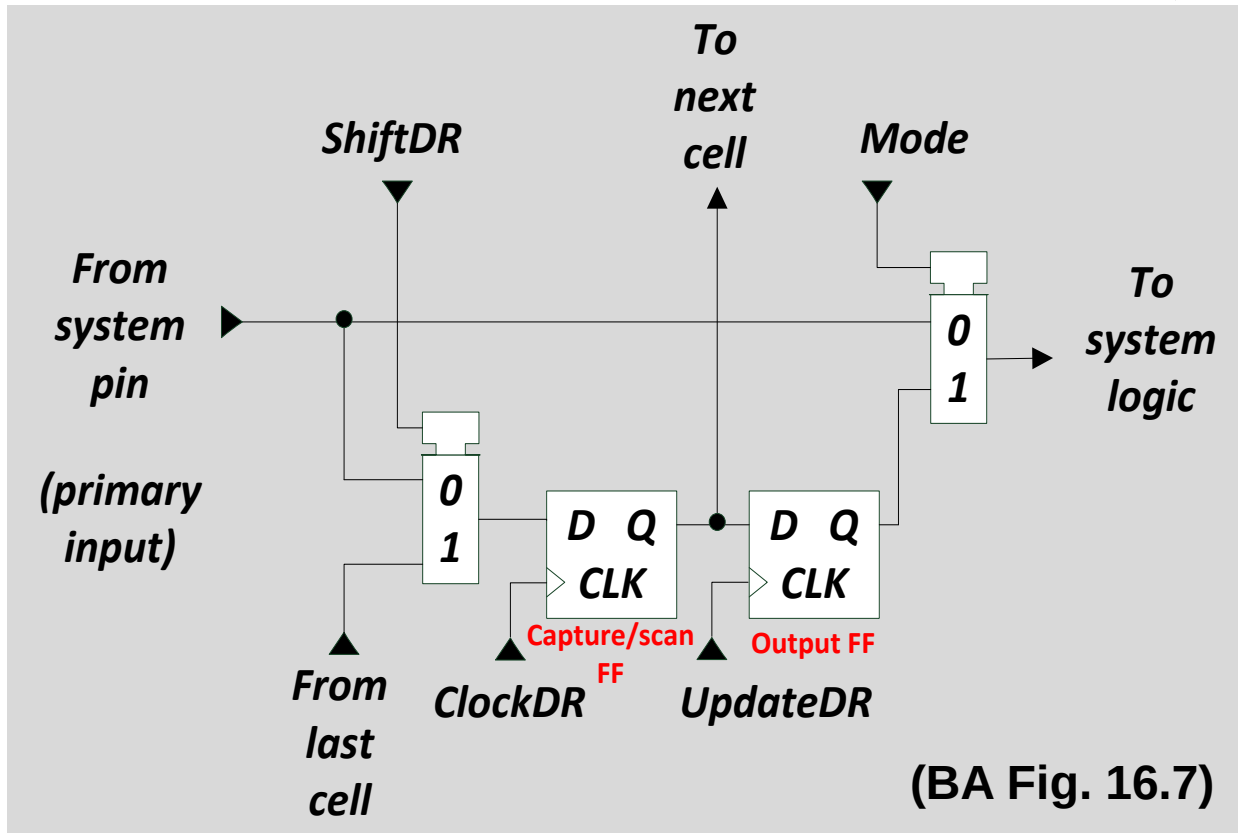
Input BSC, Output BSC



BSR Forms Boundary Scan Chain

Input Boundary Scan Cell, BSC

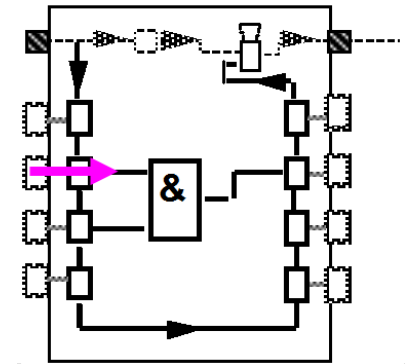
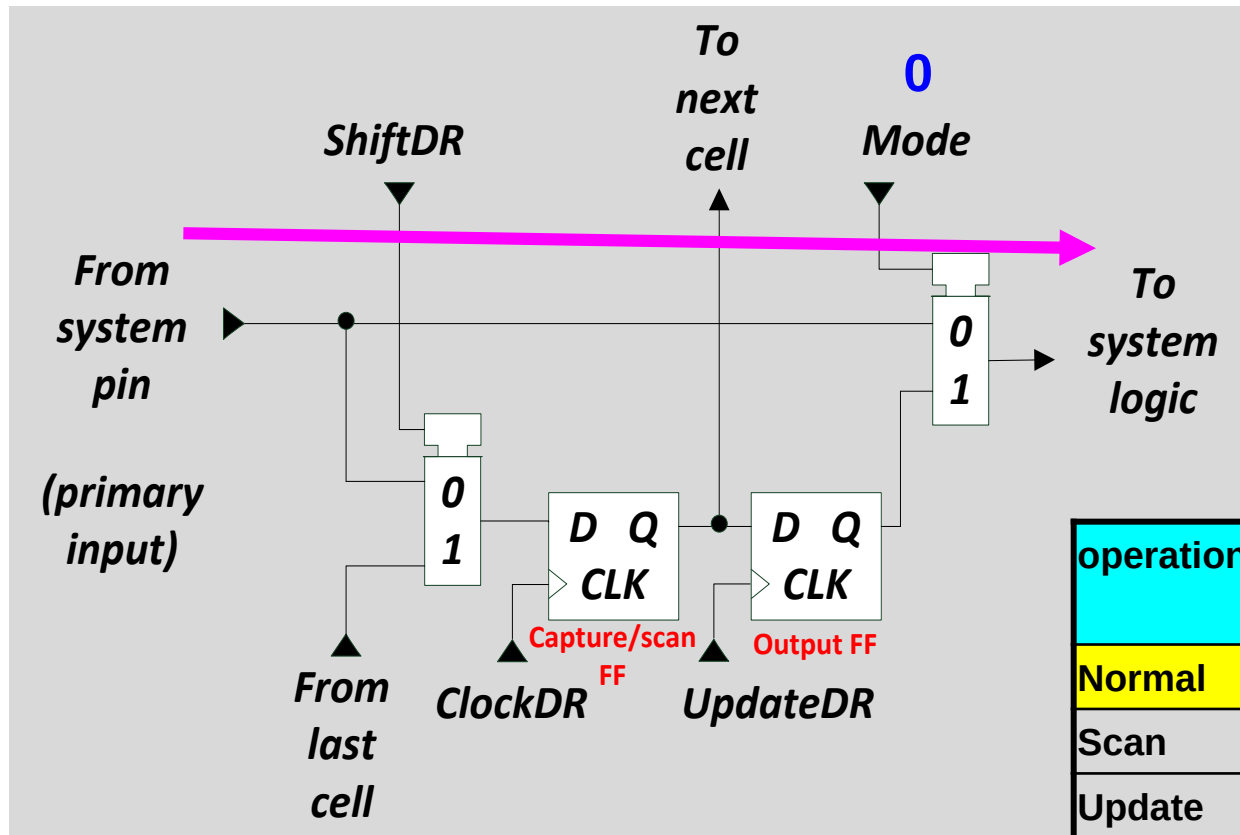
- Each input BSC has
 - 2 FF: capture/scan FF, output FF
 - 2 control signals: ShiftDR, Mode
 - 2 clocks: ClockDR, UpdateDR



**BSC Supports
4 Operations**

Input BSC Operation #1: Normal

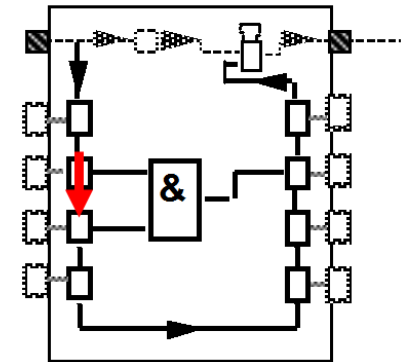
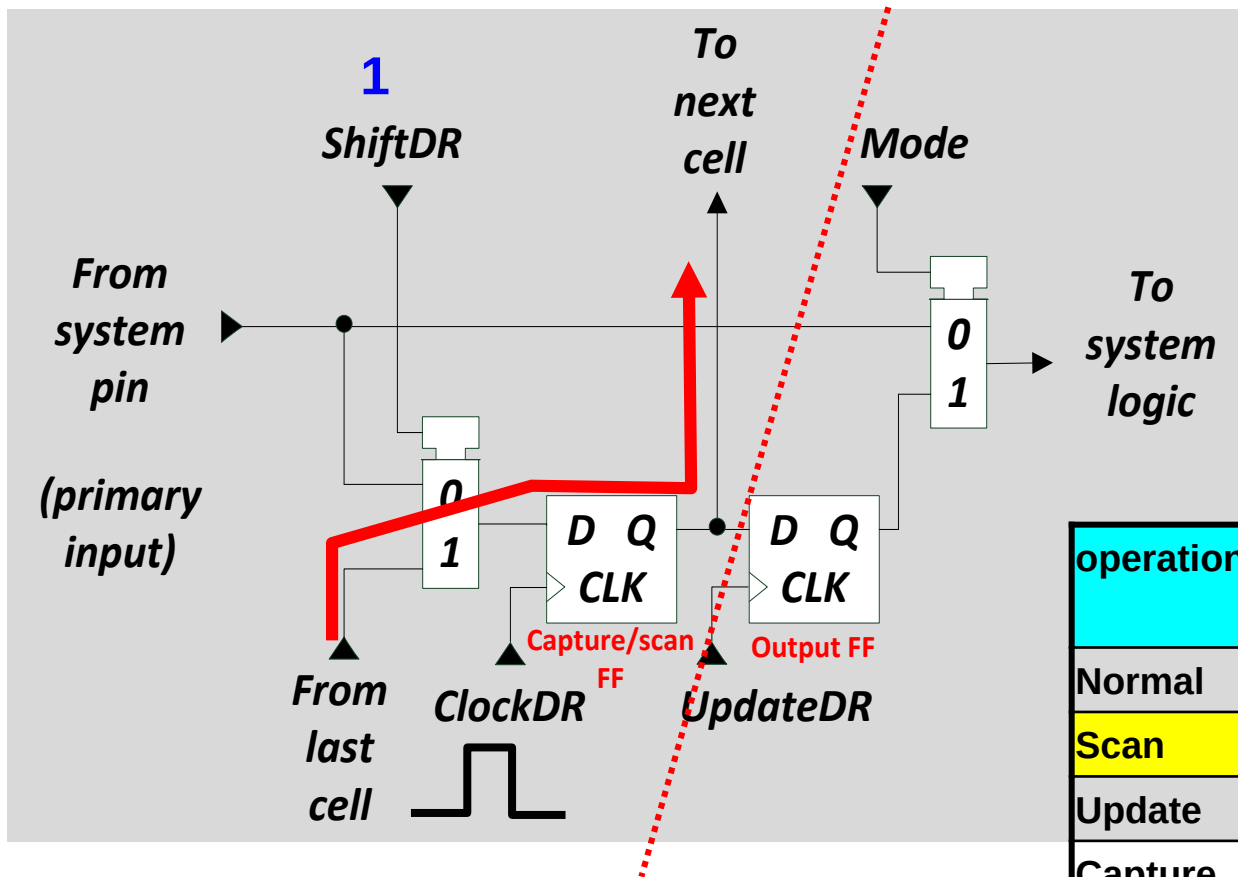
- From **system pin** to **system logic**
- BSC is **transparent**



operation	Contr'l Signal		Clock
	Mode	ShiftDR	
Normal	0	X	system
Scan	X	1	ClockDR
Update	1	X	UpdateDR
Capture	X	0	ClockDR

Input BSC Operation #2: Scan

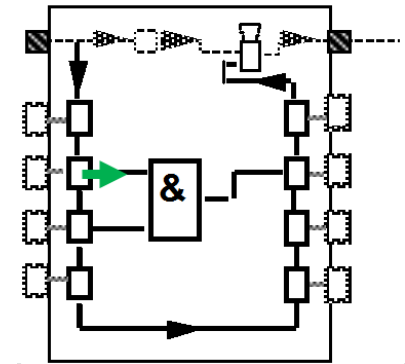
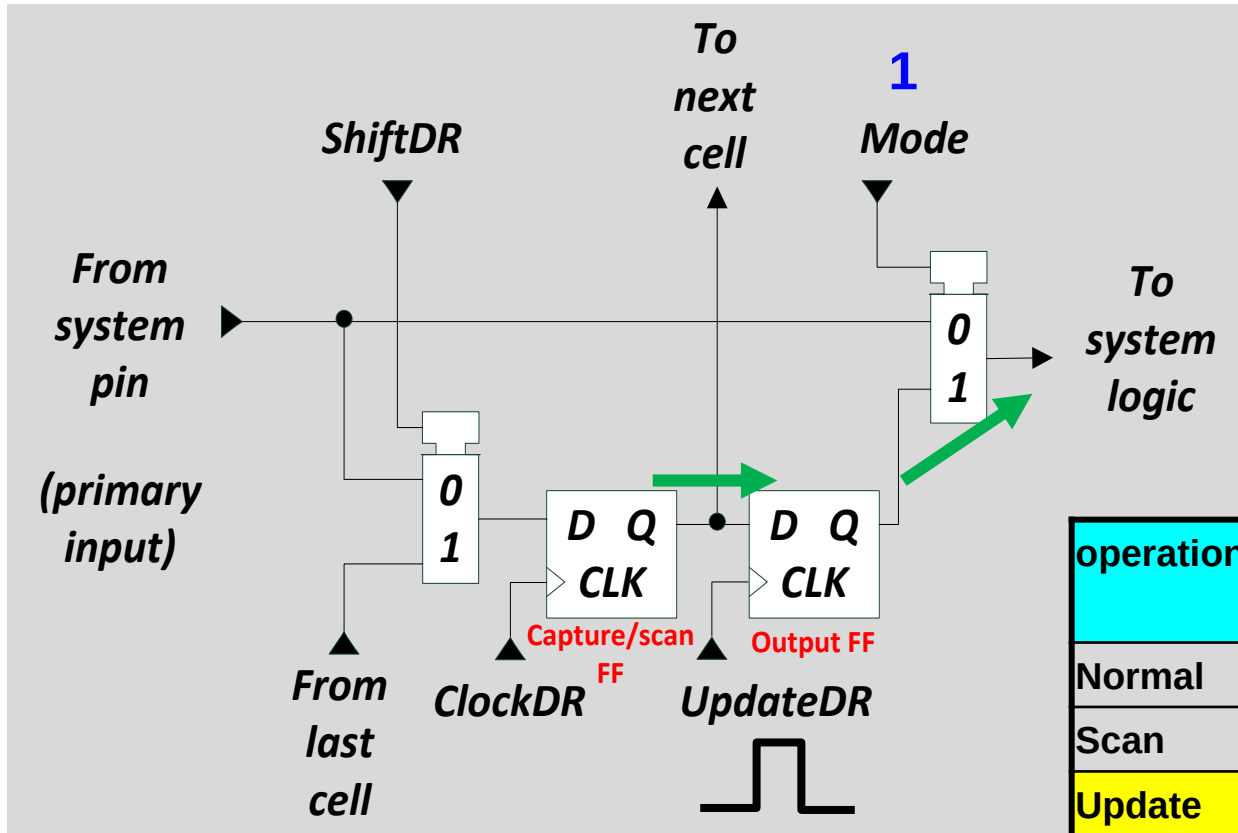
- Shift from **one Scan FF** to **next scan FF**
- Scan **does NOT interfere** with system logic



operation	Contr'l Signal		Clock
	Mode	ShiftDR	
Normal	0	X	system
Scan	X	1	ClockDR
Update	1	X	UpdateDR
Capture	X	0	ClockDR

Input BSC Operation #3: Update

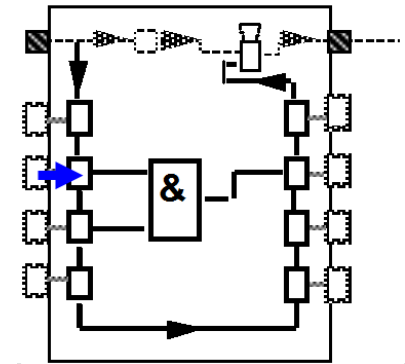
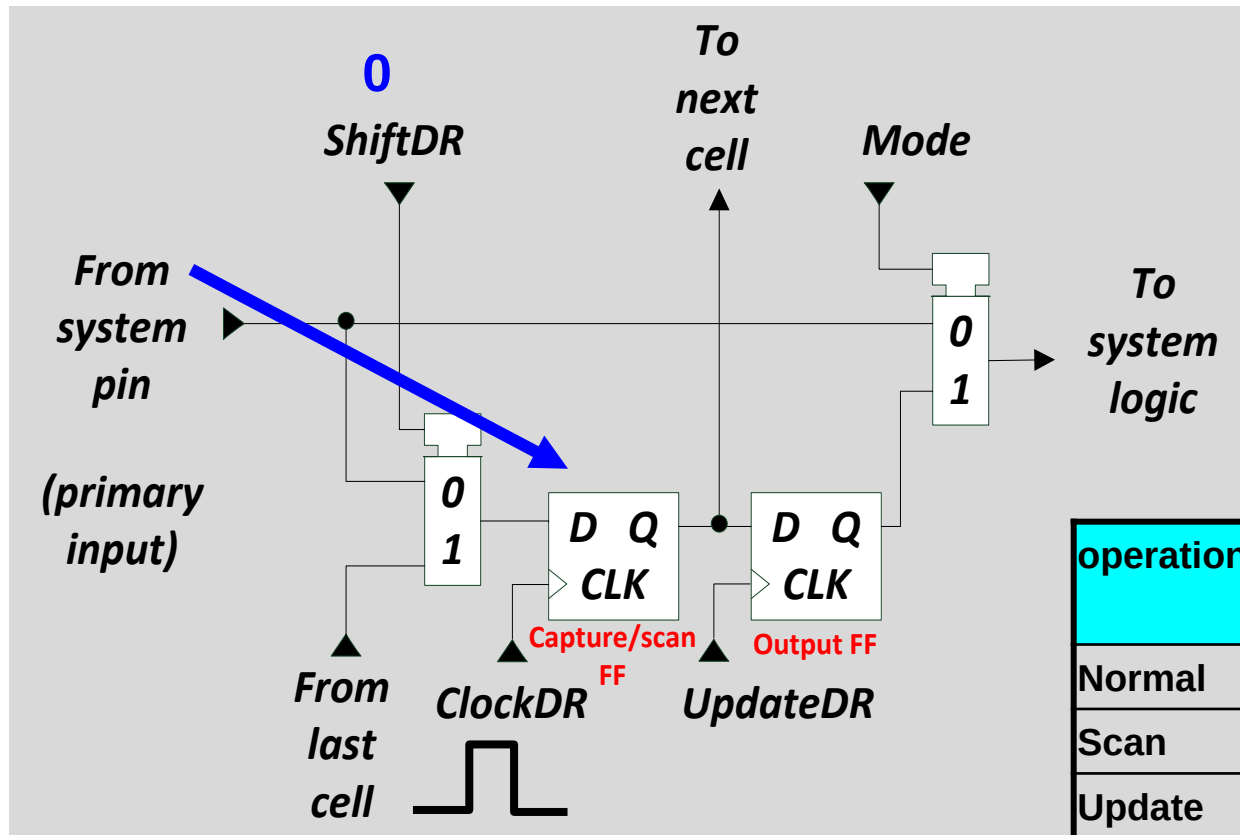
- Load data from **scan FF** to **output FF**
- **Apply test pattern** to system logic



operation	Contr'l Signal		Clock
	Mode	ShiftDR	
Normal	0	X	system
Scan	X	1	ClockDR
Update	1	X	UpdateDR
Capture	X	0	ClockDR

Input BSC Operation #4: Capture

- Capture test responses
 - ♦ From system input to capture FF



operation	Contr'l Signal		Clock
	Mode	ShiftDR	
Normal	0	X	system
Scan	X	1	ClockDR
Update	1	X	UpdateDR
Capture	X	0	ClockDR

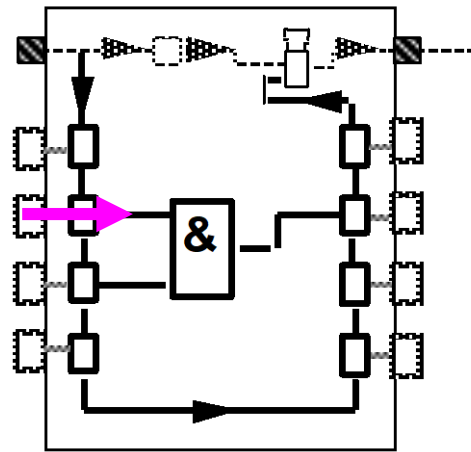
Summary of BSC Operations

same color used in 12.3~12.4

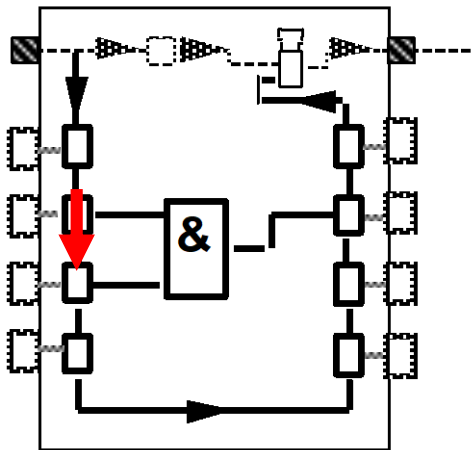
red=scan

green=update

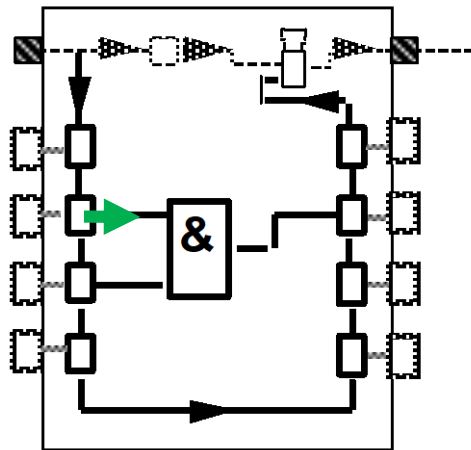
blue=capture



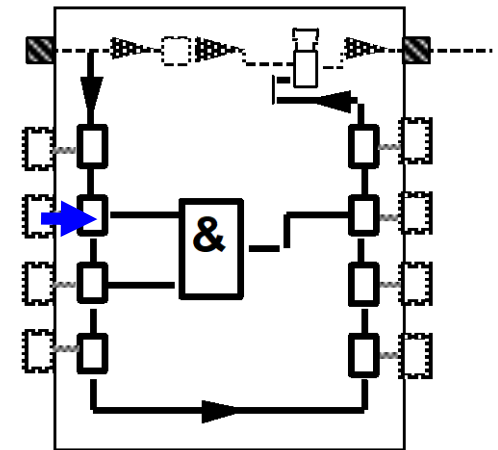
Normal



Scan



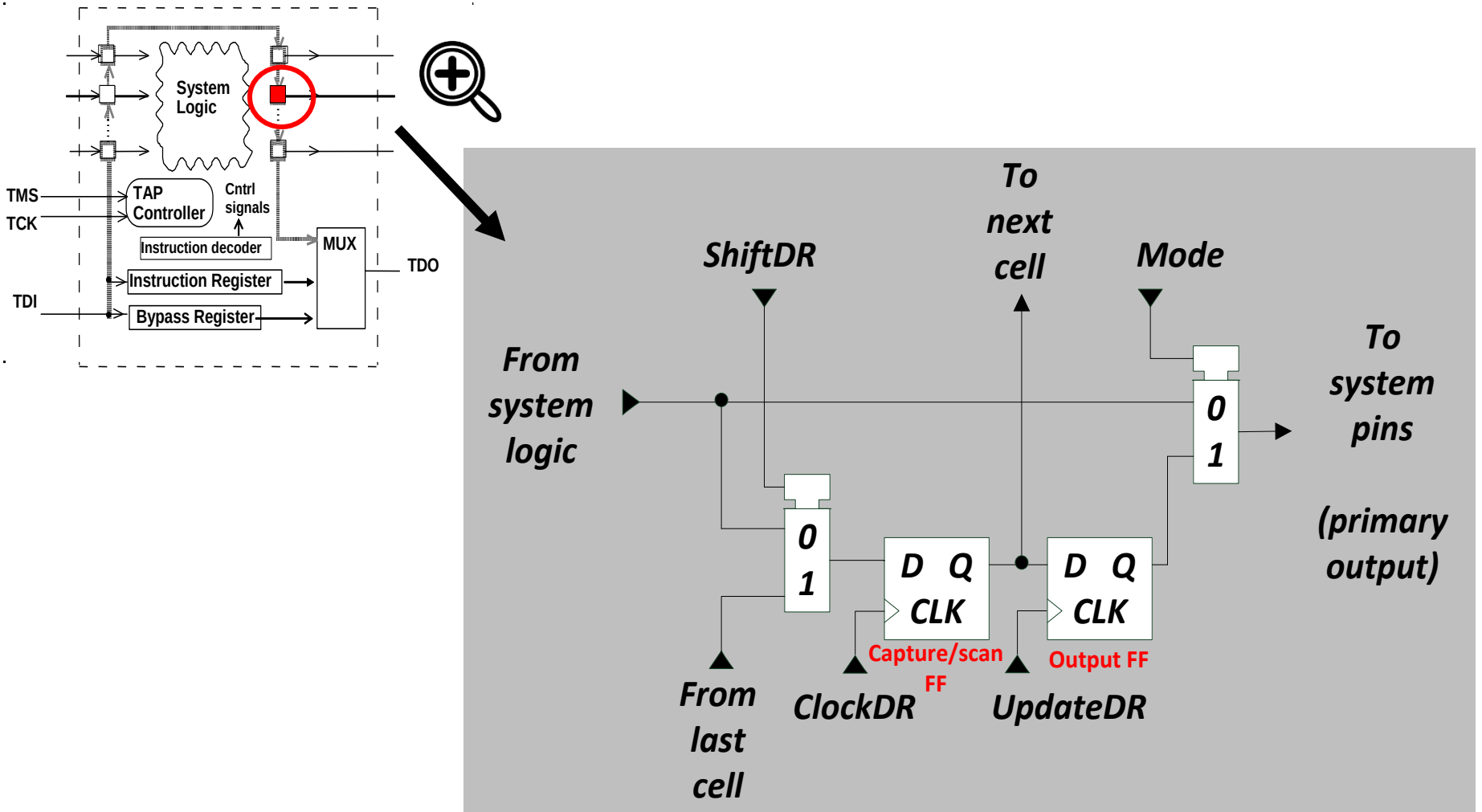
Update



Capture

Output BSC

- Very similar structure to input BSC, but different direction

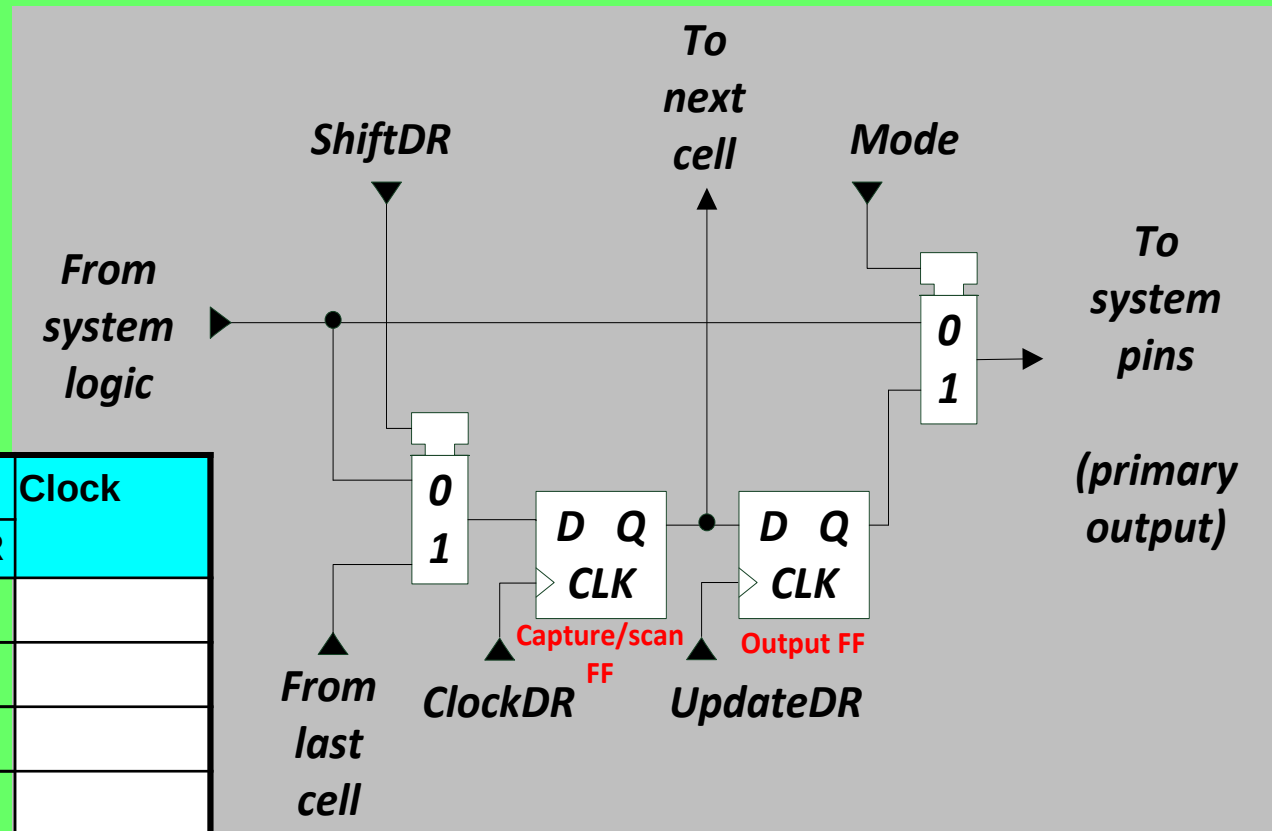


Quiz

Q: For output BSC, please fill in table for four operations

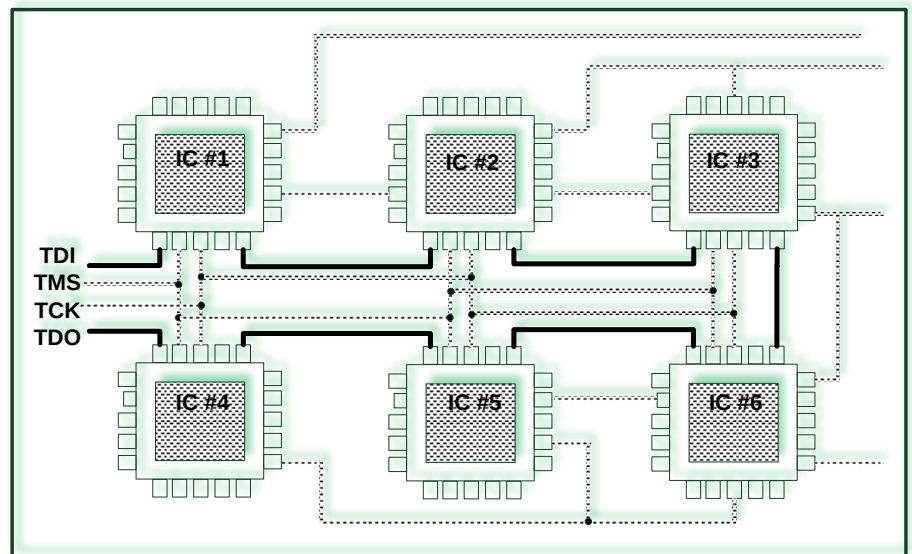
A:

operation	Contr'l Signal		Clock
	Mode	ShiftDR	
Normal	0	X	
Scan	X	1	
Capture	X	0	
Update	1	X	



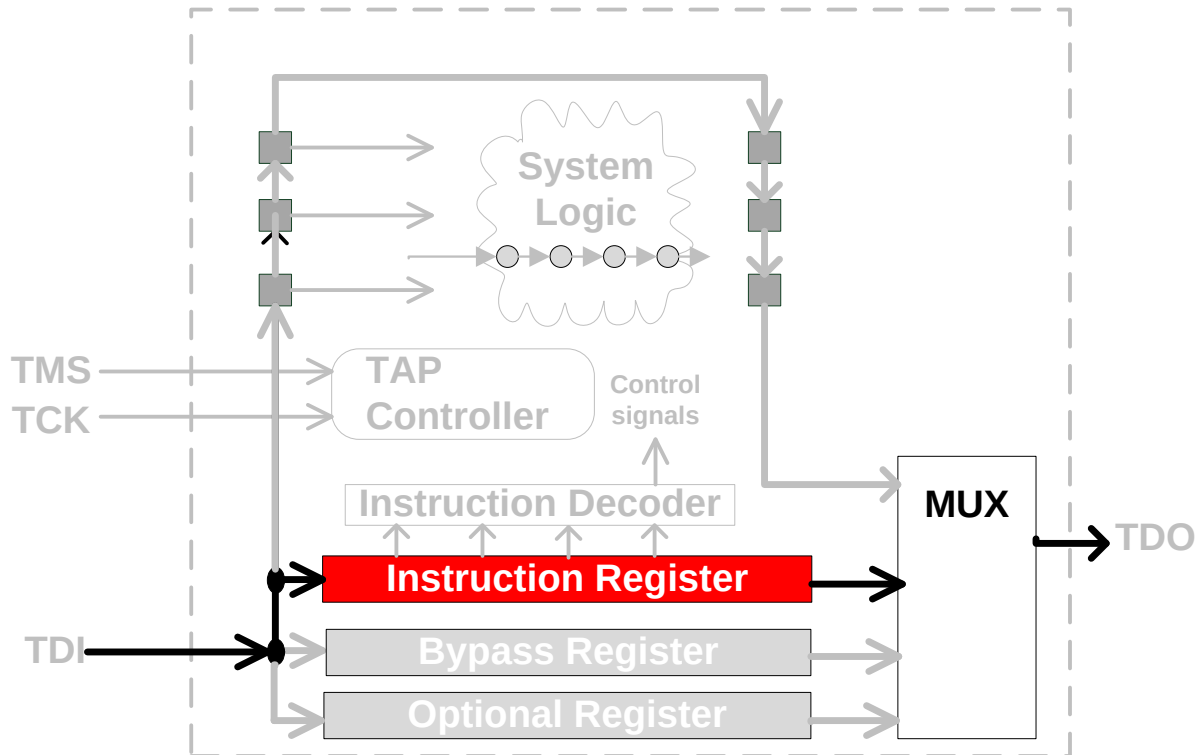
DFT – Part 2

- Introduction
- JTAG Architecture and Components
 - ◆ TAP
 - ◆ TAP controller
 - ◆ Registers
 - * Bypass Register (BR)
 - * Boundary Scan Register (BSR)
 - * Instruction Register (IR)
 - ◆ Instruction Decoder
- JTAG Instructions
- Conclusion



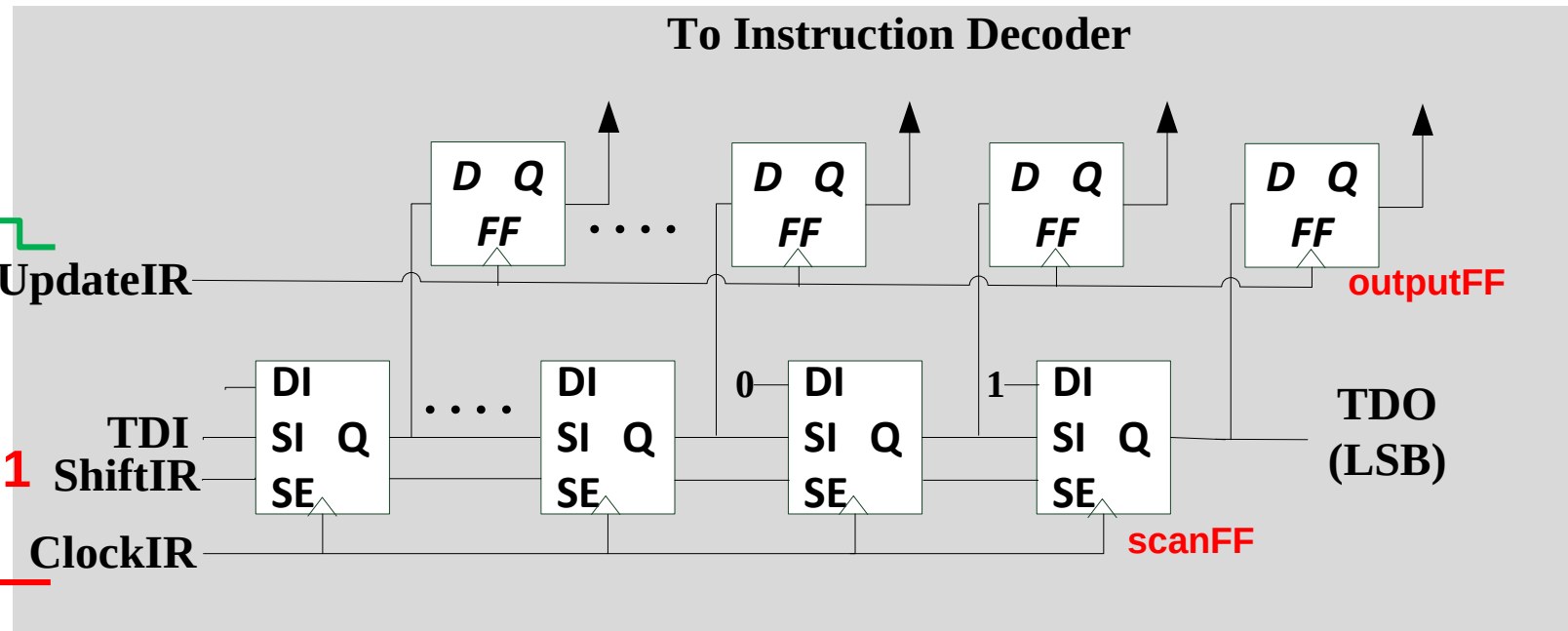
Instruction Register (1/2)

- Purposes:
 - ♦ Shift in instruction from TDI
 - ♦ Store JTAG instructions for instruction decoder



Instruction Register (2/2)

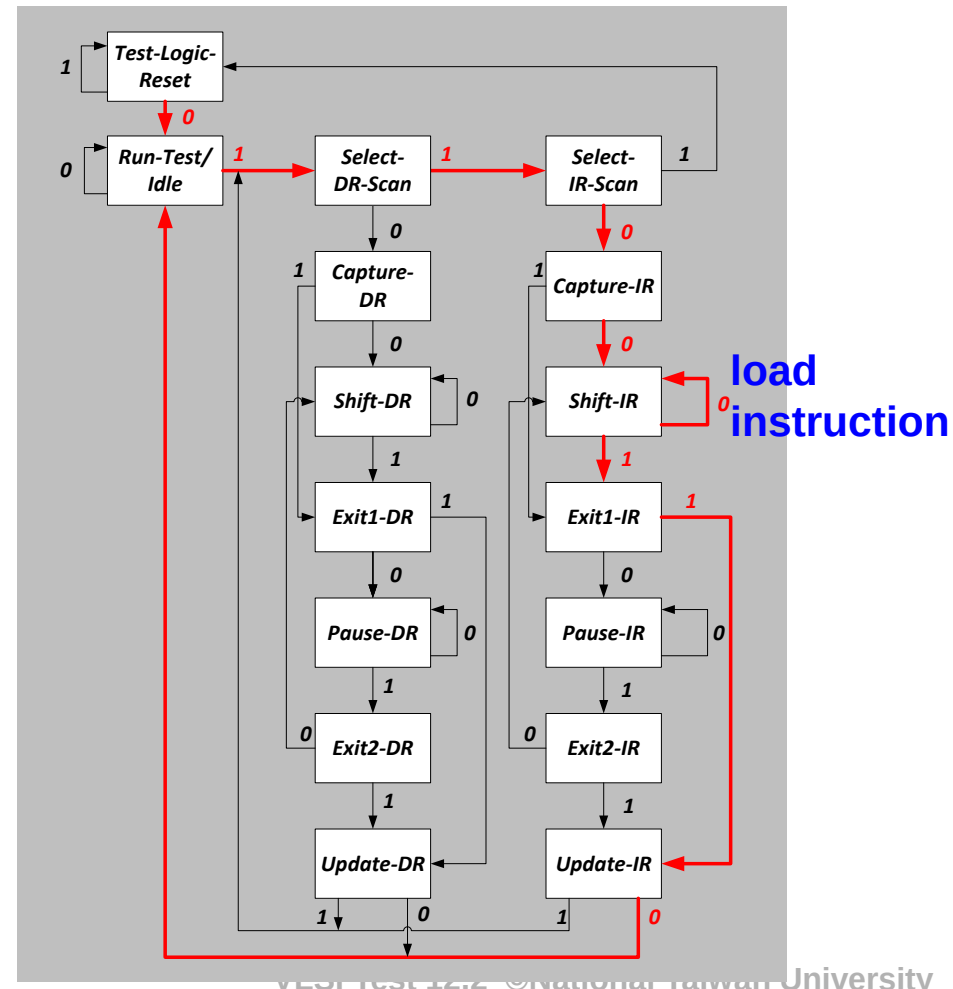
- Two layers of FF: scan FF and output FF
- How to load instruction?
 - ♦ ShiftIR=1, clock IR, clock IR
 - ♦ UpdateIR
- Scan does NOT interfere with instruction decoder



*JTAG standard requires last two bits of parallel load = '01'

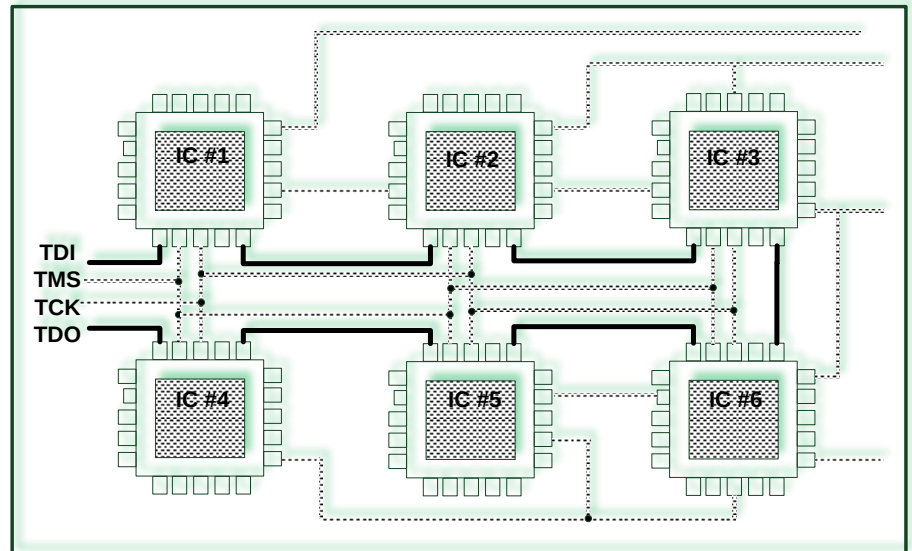
How to Load Instruction?

- **Initialize JTAG:** TMS = $1 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow 1$
 - ♦ Test-Logic-Reset state (regardless of initial state)
- **Select IR:** TMS = $0 \rightarrow 1 \rightarrow 1 \rightarrow 0 \rightarrow 0$
 - ♦ ShiftIR state
- **Load instruction:** TMS = $0 \dots 0$
 - ♦ keep in ShiftIR state
 - ♦ Instruction shifted in via **TDI**
- **Finish:** TMS = $1 \rightarrow 1 \rightarrow 0$
 - ♦ back to Run-Test-Idle state



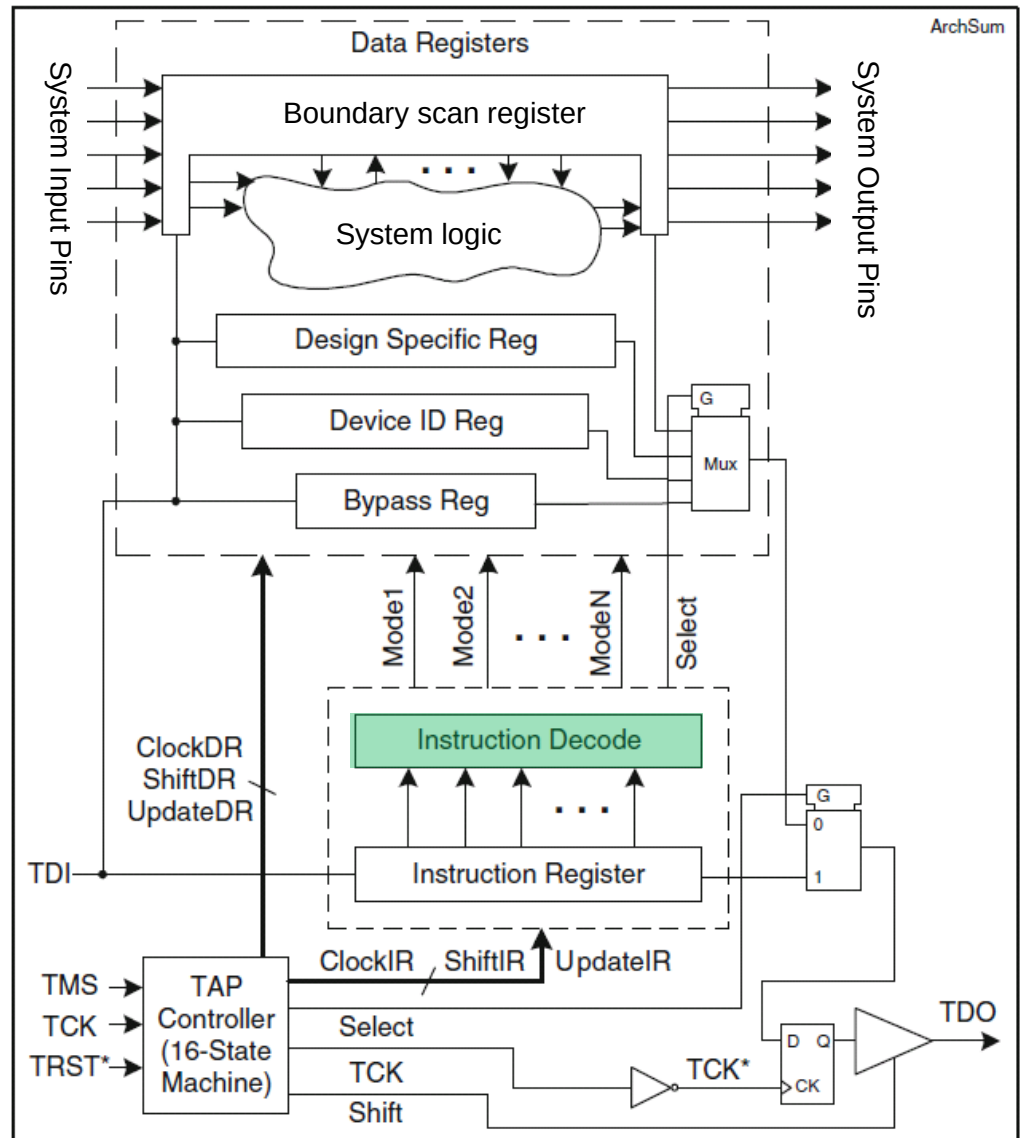
DFT – Part 2

- Introduction
- JTAG Architecture and Components
 - ◆ TAP
 - ◆ TAP controller
 - ◆ Registers
 - ◆ Instruction Decoder
- JTAG Instructions
- Conclusion



Instruction Decoder

- **Instruction decoder generates control signals**
 - ◆ **Mode**
 - ◆ **Select (DR)**
- JTAG Timing (not in exam)
- *Rising edge of TCK
 - ◆ ClockDR, ClockIR
 - ◆ TAP state transition
- *Falling edge of TCK
 - ◆ UpdateDR, UpdateIR
 - ◆ TDO output



Summary

- **Data Register (DR)**
 - ◆ **Bypass register (BR)**
 - ◆ **Boundary scan register (BSR)**
 - * **Input Boundary Scan Cell**
 - * **Output Boundary Scan Cell**
- **Each BSC**
 - ◆ **2 controls: ShiftDR, Mode**
 - ◆ **2 clocks: ClockDR, UpdateDR**
 - ◆ **4 operations: normal, scan, capture, update**
- **Instruction register (IR)**
 - ◆ **Control signals generated by instruction decoder**

